TP8 : Graphes orientés et parcours en profondeur

CPES ≪ Sciences des données, arts et cultures ≫ : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

27 février 2024

Avant de commencer le sujet de ce TP8, terminez l'implémentation de l'algorithme de parcours en largeur commencée lors du précédent TP.

Adaptation de la classe Graphe pour les graphes orientés

Dans ce premier exercice, nous allons adapter le code de la classe Graphe afin de pouvoir implémenter des graphes orientés. En effet, la seule différence entre un graphe non-orienté et un digraphe est l'*orientation* des arêtes : dans le digraphe, chaque arête est orientée ; elle a donc une origine et une destination. Dans notre implémentation par *liste d'adjacence* du graphe, ajouter une arête entre deux sommets correspondait à ajouter le premier sommet dans la liste d'adjacence de l'autre, et vice-versa. Ainsi, nous avions déjà intrinsèquement défini le digraphe, où nous avions systématiquement deux arêtes orientés permettant de passer d'un sommet à l'autre dans les deux directions.

De ce fait, pour implémenter un graphe digraphe, il suffit de précise la (ou les) direction(s) de la (ou des) arête(s) que l'on souhaite créer entre deux sommets.

Question 1 Au sein de la classe *Graphe*, définir une méthode `ajout_arete_digraphe`, qui doit créer une ou deux arêtes entre deux sommets du digraphe. Elle prendra pour paramètres, en plus de `self` :

- `id_sommet_1`, l'ID du premier sommet.
- `id_sommet_2`, l'ID du second sommet.
- `arete_de_1_vers_2`, une variable *booléenne* décrivant si oui ou non il faut créer une arête ayant pour origine le sommet d'ID `id_sommet_1` et pour destination le sommet d'ID `id_sommet_2`.
- `arete_de_2_vers_1`, une variable *booléenne* décrivant si oui ou non il faut créer une arête ayant pour origine le sommet d'ID `id_sommet_2` et pour destination le sommet d'ID `id_sommet_1`.

Vous utiliserez pour cela la méthode `ajout_voisin` des objets représentant les deux sommets d'ID `id_sommet_1` et `id_sommet_2`.

Question 2 Tester la correction de votre implémentation des digraphes en créant un petit digraphe, et affichez-le en utilisant la méthode magique `__str__` définie dans le TP7 (pour rappel, elle est fournie dans le fichier `code_fourni.py`, disponible sur le Moodle temporaire du cours). Est-il toujours possible d'implémenter des graphes non-orientés en utilisant la fonction `ajout_arete_digraphe` ? Justifier.

Implémentation du parcours en profondeur

Dans cet exercice, nous allons implémenter la version *récursive* de l'algorithme de parcours en profondeur étudié en cours. De la même façon que nous l'avons fait dans le TP7, nous allons implémenter une nouvelle classe *SommetDFS*, héritante de la classe *Sommet*. Elle implémentera les attributs déjà utilisés dans `SommetBFS`, à savoir une couleur, une distance et une parentalité, ainsi que des attributs supplémentaires :

— un attribut *temps de découverte*, qui sera noté `d`, représentant le moment auquel le sommet a été découvert (donc grisé).

— un attribut *temps de fin*, qui sera noté `f`, représentant le moment auquel le sommet a été "fini" (donc noirci).

Question 3 Implémenter une classe *SommetDFS*, héritante de la classe *Sommet*, dans laquelle vous écrirez deux fonctions. Un constructeur, qui initialise par défaut les valeurs des attributs décrits plus haut, et une méthode `initialisation_dfs`, préparant les attributs de l'objet pour un nouveau parcours en profondeur. Vous pourrez vous inspirer de ce qui a été fait dans le TP7 pour la classe *SommetBFS* pour écrire cette nouvelle classe qui en est très proche.

Question 4 A partir du pseudo-code de l'algorithme de parcours en profondeur *récursif* fourni dans le cours à la slide 22, écrire :

— Une fonction `parcours_en_profondeur`, prenant en paramètres un objet *Graphe*, nommé `graphe`, qu'on supposera constitué de sommets *SommetDFS* (donc, on supposera que l'on aura défini le paramètre `classe_sommet = SommetDFS` lors de la construction du graphe). Cette fonction correspondra à la fonction DFS(V) dans le cours.

— Une fonction `parcours_en_profondeur_sommet`, prenant en paramètres un objet *Graphe*, nommé `graphe`, qu'on supposera constitué de sommets *SommetDFS* (donc, on supposera que l'on aura défini le paramètre `classe_sommet = SommetDFS` lors de la construction du graphe), et un entier `id_sommet`, représentant l'ID du sommet à partir duquel commencer le parcours en profondeur. Cette fonction correspondra à la fonction DFS-VISITE(D, v) dans le cours.

Question 5 Une fois les deux fonctions implémentées, les compléter pour pouvoir observer les différentes étapes du parcours. Il vous faudra pour cela, à chaque itération du parcours, afficher le numéro de l'itération et l'`id` du sommet visité. Testez ensuite votre fonction sur la graphe K_4 construit dans le TP7, et sur un graphe généré aléatoirement si le temps le permets (le code permettant de générer un tel graphe est présent dans le fichier `graphe_aleatoire.py` **sur le Moodle du cours, dans le dossier "TD-TP/8/"**).